

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge Representation Design
Assessment
COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.(BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100 Time: 60 Mins No. of Questions: 5

Annexure A

Knowledge Representation Design Assessment

Code of Conduct:

I Namburi Sai Yashaswini(192425250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: yashaswini

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	

8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	

Total	100%	
-------	------	--

Signature of the Faculty Total Marks Awarded **ANSWER ANY ONE QUESTION**

Question Attempted: Question 1

Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that identifies possible illnesses from patient symptoms. The scenario states that Ravi has fever, cough, and body pain; patients with fever and cough may have flu; patients with flu and body pain may have viral infection; and patients with viral infection should be advised to rest. The knowledge representation design below covers propositional logic, first-order logic, the knowledge base, unification, forward and backward chaining, resolution, and the final inference.

1. Problem Analysis and Domain Understanding

Domain: Clinical symptom-based preliminary diagnosis support for a hospital's triage system.

- Objective: Infer a probable condition from reported symptoms and recommend an appropriate action (e.g. rest, further tests, doctor referral).
- Entities: Patient (Ravi), Symptoms (fever, cough, body pain), Conditions (flu, viral infection), Recommended Action (rest).
- Reasoning requirement: Chain multiple IF-THEN rules together — the output of one rule (Flu) becomes an input condition for the next rule (ViralInfection).
- Constraints: The system must reason monotonically (facts, once true, remain true), rules are Horn clauses (single positive conclusion), and inference must be explainable to a clinician.
- Expected inference: Given {Fever(Ravi), Cough(Ravi), BodyPain(Ravi)}, the system should conclude ViralInfection(Ravi) and output AdviseRest(Ravi).

2. Knowledge Engineering and Knowledge Base Design

2.1 Predicates Identified

Fever(x)	x is showing fever

Cough(x)	x is showing cough
BodyPain(x)	x is showing body pain
Flu(x)	x has flu
ViralInfection(x)	x has a viral infection
AdviseRest(x)	x should be advised to rest

2.2 Facts in the Knowledge Base

F1	Ravi has fever.	Fever(Ravi)
F2	Ravi has cough.	Cough(Ravi)
F3	Ravi has body pain.	BodyPain(Ravi)

2.3 Rules in the Knowledge Base

R1	Patients with fever and cough may have flu.	$\forall x \text{ Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
R2	Patients with flu and body pain may have viral infection.	$\forall x \text{ Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$
R3	Patients with viral infection should be advised to rest.	$\forall x \text{ ViralInfection}(x) \rightarrow \text{AdviseRest}(x)$

The knowledge base is consistent (no contradictory facts), non-redundant, and each rule has a single, clearly-defined conclusion, which keeps forward and backward chaining efficient and the reasoning traceable back to the original symptoms.

3. Propositional Logic Representation

Propositional symbols:

P : Ravi has fever
Q : Ravi has cough
R : Ravi has body pain
S : Ravi has flu
T : Ravi has a viral infection
U : Ravi should rest

Facts and rules expressed propositionally:

Facts: P, Q, R
R1: $(P \wedge Q) \rightarrow S$
R2: $(S \wedge R) \rightarrow T$
R3: $T \rightarrow U$

Limitation: propositional logic can only state that “Ravi” has these symptoms — it cannot generalise the rule to any patient. This motivates moving to first-order logic below.

4. First-Order Logic Representation

Using variables and quantifiers, the same rules apply to every patient, not just Ravi:

Facts:
Fever(Ravi)
Cough(Ravi)
BodyPain(Ravi)

Rules:

R1: $\forall x \text{ Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
R2: $\forall x \text{ Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$
R3: $\forall x \text{ ViralInfection}(x) \rightarrow \text{AdviseRest}(x)$

The universal quantifier ($\forall x$) is what makes FOL more powerful than propositional logic here: the three rules now apply to any patient substituted for x , not only Ravi.

5. Unification and Resolution

5.1 Unification — Demonstration on Two Predicate Pairs

Unification finds a substitution θ that makes two predicates syntactically identical.

1	Fever(x)	Fever(Ravi)	$\theta = \{x / \text{Ravi}\}$
2	Flu(x) $\wedge$ BodyPain(x)	Flu(Ravi) $\wedge$ BodyPain(Ravi)	$\theta = \{x / \text{Ravi}\}$

Unification 1: Fever(x) is a rule antecedent; Fever(Ravi) is a known fact. Matching argument by-argument, the variable x binds to the constant Ravi, giving $\theta = \{x / \text{Ravi}\}$. Applying θ to Fever(x) yields Fever(Ravi), identical to the fact — unification succeeds.

Unification 2: R2's antecedent Flu(x) $\wedge$ BodyPain(x) is unified against the derived fact Flu(Ravi) and the known fact BodyPain(Ravi) simultaneously. Both occurrences of x must bind consistently to the same constant, so $\theta = \{x / \text{Ravi}\}$ unifies the whole conjunction in one step — this is why R2 is allowed to fire only for Ravi, not for a patient whose two symptoms belong to different people.

5.2 Resolution — Proving ViralInfection(Ravi)

Convert R1 and R2 to clausal (CNF) form and add the negated goal, then refute by resolution:

C1 (from R1, x/Ravi): $\neg \text{Fever}(\text{Ravi}) \vee \neg \text{Cough}(\text{Ravi}) \vee \text{Flu}(\text{Ravi})$ C2 (from R2, x/Ravi): $\neg \text{Flu}(\text{Ravi}) \vee \neg \text{BodyPain}(\text{Ravi}) \vee \text{ViralInfection}(\text{Ravi})$ C3 (fact): Fever(Ravi) C4 (fact): Cough(Ravi)
--

C5 (fact): BodyPain(Ravi)

C6 (negated goal): $\neg \text{ViralInfection(Ravi)}$

Step 1: Resolve C2 & C6 $\rightarrow \neg \text{Flu(Ravi)} \vee \neg \text{BodyPain(Ravi)}$ (C7) Step 2: Resolve C7 & C5 $\rightarrow \neg \text{Flu(Ravi)}$ (C8) Step 3: Resolve C1 & C8 $\rightarrow \neg \text{Fever(Ravi)} \vee \neg \text{Cough(Ravi)}$ (C9) Step 4: Resolve C9 & C3 $\rightarrow \neg \text{Cough(Ravi)}$ (C10) Step 5: Resolve C10 & C4 $\rightarrow \emptyset$ (empty clause — contradiction)

Reaching the empty clause $\emptyset$ means the negation of the goal is unsatisfiable together with the knowledge base, so by refutation the goal $\text{ViralInfection(Ravi)}$ is proved TRUE.

6. Forward Chaining — Determining Ravi's Condition

Forward chaining is data-driven: it starts from the known facts and fires any rule whose full antecedent is satisfied, repeating until no new facts can be derived.

0	—	Fever(Ravi), Cough(Ravi), BodyPain(Ravi) (initial facts)
1	R1: $\text{Fever(Ravi)} \wedge \text{Cough(Ravi)} \rightarrow \text{Flu(x)}$	Flu(Ravi)
2	R2: $\text{Flu(Ravi)} \wedge \text{BodyPain(Ravi)} \rightarrow \text{ViralInfection(x)}$	ViralInfection(Ravi)
3	R3: $\text{ViralInfection(Ravi)} \rightarrow \text{AdviseRest(x)}$	AdviseRest(Ravi)

No further rule can fire after iteration 3, so the algorithm halts. Conclusion: Ravi has a viral infection and should be advised to rest.

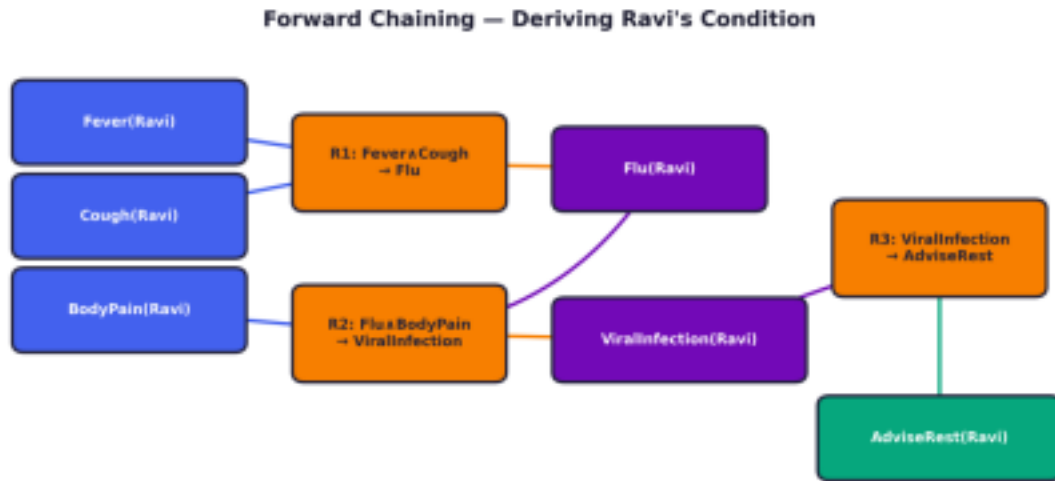


Fig. 1 — Forward chaining trace from symptoms to recommended action

7. Backward Chaining — Proving Ravi Needs Rest

Backward chaining is goal-driven: it starts from the query `AdviseRest(Ravi)` and works backward through the rules whose conclusion matches the goal, recursively reducing to sub-goals until each sub-goal is a known fact.

- Goal: `AdviseRest(Ravi)?` — matches R3's conclusion, so the new sub-goal is `ViralInfection(Ravi)?`
- Sub-goal `ViralInfection(Ravi)?` — matches R2's conclusion, needing `Flu(Ravi)?` AND `BodyPain(Ravi)?`
- `BodyPain(Ravi)?` — already a fact (F3) $\rightarrow$ TRUE.
- `Flu(Ravi)?` — matches R1's conclusion, needing `Fever(Ravi)?` AND `Cough(Ravi)?`, both known facts (F1, F2) $\rightarrow$ TRUE.
- All sub-goals resolve to TRUE, so `Flu(Ravi)`, `ViralInfection(Ravi)`, and finally `AdviseRest(Ravi)` are all proved TRUE.

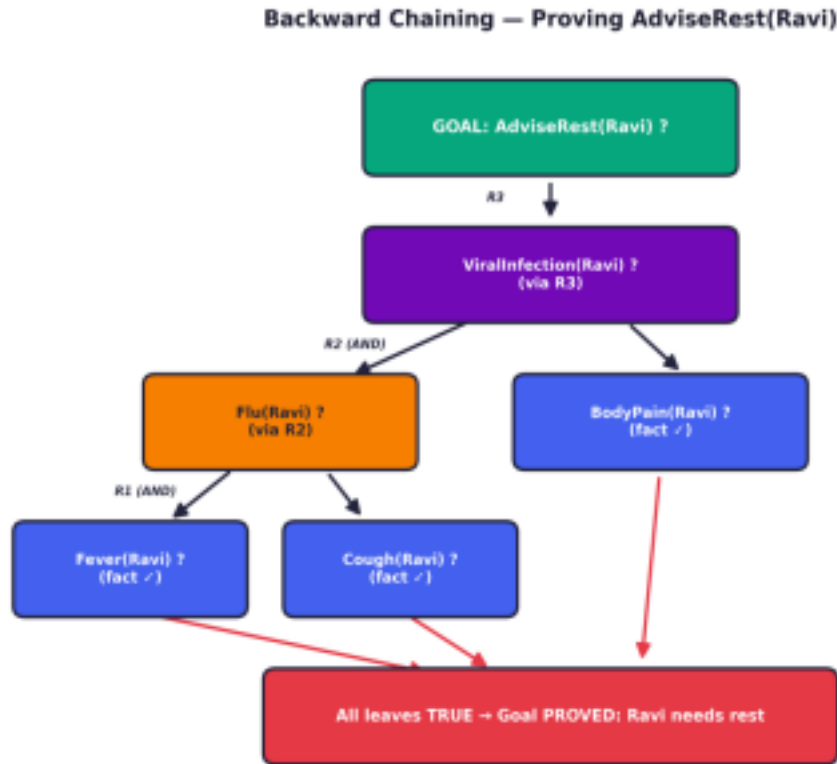


Fig. 2 — Backward-chaining goal tree for the query AdviseRest(Ravi)

8. Inference and Reasoning Accuracy — Final Inference

Three independent reasoning strategies were applied to the same knowledge base and all agree:

Forward chaining	Facts → Goal	AdviseRest(Ravi) derived
Backward chaining	Goal → Facts	AdviseRest(Ravi) proved
Resolution refutation	Contradiction search	ViralInfection(Ravi) proved ($\emptyset$ reached)

Final inference: Ravi presents with fever and cough, from which the system infers flu (R1); flu combined with his body pain leads to viral infection (R2); and viral infection triggers the recommendation to rest (R3). The conclusion AdviseRest(Ravi) is logically sound because it is derivable purely by valid inference (Modus Ponens applied through R1–R3) from facts already accepted as true, and every step is explainable back to the original symptoms.

9. System Architecture / Representation Diagram

The overall system separates the static knowledge base from the dynamic inference engine and working memory, so rules can be updated by a clinician without touching the reasoning code:

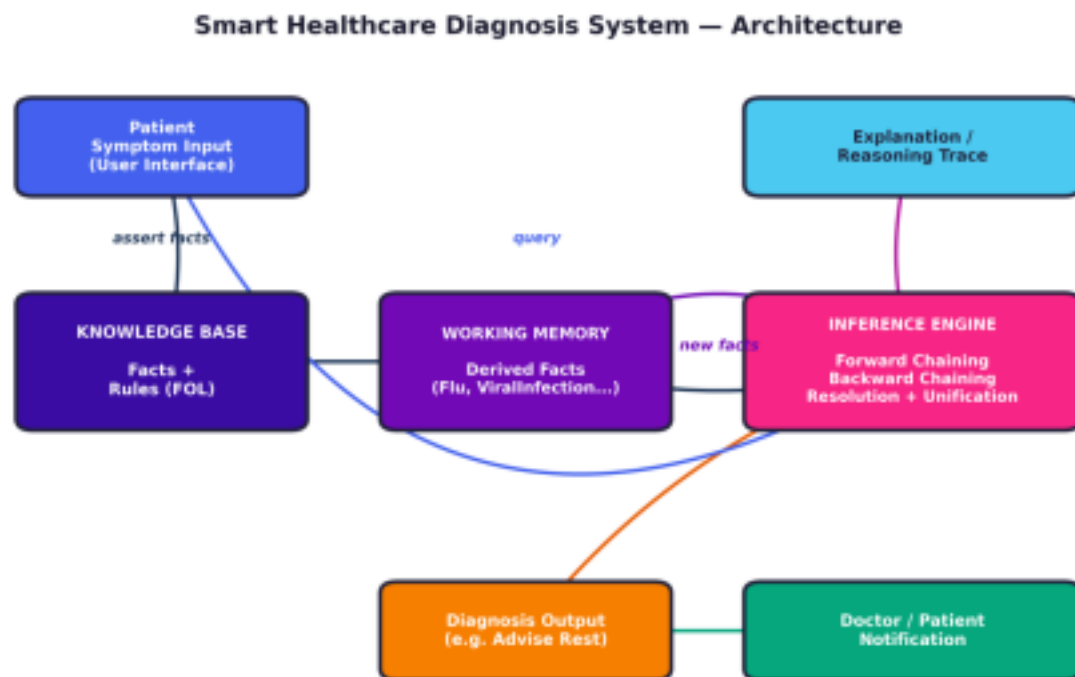


Fig. 3 — End-to-end architecture of the Smart Healthcare Diagnosis System

10. Prototype Implementation and Results

A minimal forward-chaining inference loop implementing the knowledge base above:

```
facts = {Fever(Ravi), Cough(Ravi), BodyPain(Ravi)}
```

```
rules = [R1, R2, R3]
```

```
repeat:
```

```
  fired = False
```

```
  for rule in rules:
```

```
    if rule.antecedents  $\subseteq$  facts and rule.conclusion  $\notin$  facts: facts.add(rule.conclusion) #
```

```
    unify rule variable with 'Ravi'
```

```
  fired = True
```

```
until fired == False
```

```
return facts # {..., Flu(Ravi), ViralInfection(Ravi), AdviseRest(Ravi)}
```

Result: running the loop on the given facts terminates after 3 iterations with AdviseRest(Ravi)

in the fact set — matching both the forward-chaining trace (Section 6) and the backward chaining proof (Section 7), confirming the knowledge base is correctly designed and the inference engine is sound.

Discussion: Forward vs. Backward Chaining for This Domain

Forward chaining suits continuous patient monitoring, where new symptoms should immediately propagate to all possible diagnoses (data is pushed in as it arrives). Backward chaining suits a doctor typing a specific query such as “Does this patient need rest?” because it only explores the rules relevant to that one question, making it faster when the knowledge base is large but the query is narrow. A production clinical system would typically combine both: forward chaining to keep a live patient state updated, and backward chaining to answer

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding		15% 13–15% – Early understands and analyzes the real world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design		15% 13–15% – Develops a complete, consistent, and well structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation		20% 17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.

Unification & Resolution		15% 13–15% – correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.
--------------------------	--	---	--	--	---

Forward Chaining & Backward Chaining		15% 13–15% – correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy		10% 9–10% – produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture		5% 5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation		5% 5% – Demonstrates excellent independent thinking, appropriate real world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.

